



TECHNICAL DOCUMENTATION

FMEAD-FPGA

FPGA Multi-Modal Edge Anomaly Detector

Author

Amilton Domingos Alexandre Koxi

Platform

Digilent Nexys 4 DDR, Xilinx Artix-7 FPGA

Tools

Vivado, Vitis, MicroBlaze, Bare-metal C, AXI Peripherals

Project Type

FPGA Embedded Systems Prototype

Date

May 2026



Abstract

This report presents the design and implementation of FMEAD-FPGA, a multi-modal edge anomaly detector implemented on a Digilent Nexys 4 DDR board with an Artix-7 FPGA. The system uses a MicroBlaze soft-core processor, a set of AXI peripherals, and bare-metal C firmware developed in Vitis. It integrates four sensing paths: an ADXL362 three-axis accelerometer over SPI, a BME280 environmental sensor over I2C, the XADC internal FPGA temperature monitor, and a PDM microphone front-end exposed to software through an AXI GPIO interface.

The objective of the project is to demonstrate a practical reconfigurable embedded system that performs local acquisition, filtering, event classification, and visualization without relying on a desktop computer or cloud service during operation. The design reports motion events, environmental conditions, FPGA die temperature, and simple acoustic events through local outputs and a Wi-Fi dashboard served by a PmodESP32 module. The acoustic module is intentionally limited to simple event detection, mainly QUIET and CLAP / IMPACT, and does not perform speech recognition, speaker recognition, or music recognition.

The report focuses on the implemented hardware and software architecture. It explains the Vivado block design, the role of the XDC constraints, the MicroBlaze firmware structure, each sensor interface, the local displays, the web dashboard, the testing results, the problems encountered during development, and the limitations of the current prototype.

Table of Contents

Abstract.....	2
Table of Contents	3
1. Introduction	5
2. Project Motivation and Objectives	5
3. Hardware Platform	6
3.1 Digilent Nexys 4 DDR Board.....	6
3.2 MicroBlaze Soft-Core Processor	6
3.3 Sensor and Output Modules.....	6
3.4 Prepared PmodBT2 Bluetooth Interface.....	6
4. Vivado Hardware Design	8
4.1 Block Design Overview.....	8
4.2 Main IP Blocks	10
4.3 Clocking and Reset	10
4.4 Implementation Reports and Hardware Verification.....	10
5. XDC Pin Constraints and External Connections.....	11
5.1 Role of the XDC File	11
5.2 Practical Pin Debugging	12
6. Firmware Architecture in Vitis.....	12
6.1 Bare-Metal Firmware Structure	12
6.2 Initialization Sequence.....	13
6.3 Cooperative Scheduling.....	13
6.4 Memory Safety	13
7. Sensor Integration and Detection Logic	14
7.1 ADXL362 Motion Detection	14
7.2 BME280 Environmental Monitoring.....	14
7.3 XADC FPGA Temperature Monitoring	15
7.4 PDM Microphone Acoustic Event Detection	15
7.5 Event Fusion and Risk Levels	15

8. Output Interfaces and Dashboard	16
8.1 ESP32 Wi-Fi Dashboard.....	16
8.2 Seven-Segment Display and LEDs.....	17
8.3 Pmod OLED RGB Display.....	17
8.4 Prepared Bluetooth Telemetry Interface.....	17
9. Testing Results	17
9.1 Integrated Prototype Test.....	17
9.2 Motion Dashboard Test	18
9.3 Environmental Dashboard Test	19
9.4 Acoustic Dashboard Test.....	19
9.5 Home Dashboard Test	20
9.6 OLED and Seven-Segment Local Display Test	22
9.7 Test Summary	22
10. Problems Faced and Solutions.....	23
10.1 Sensor Bring-Up Issues	23
10.2 ESP32 and HTTP Stability	23
10.3 Seven-Segment Flicker During Communication	23
10.4 Acoustic Detector Sensitivity	23
10.5 Local Memory Pressure	24
11. Limitations.....	24
12. Future Work.....	24
13. Conclusion.....	26
References	27
Appendix Checklist	29

1. Introduction

Embedded systems are often required to react to physical events close to the point where data is generated. In applications such as equipment monitoring, environmental supervision, laboratory prototypes, and edge devices, it is not always efficient to send every raw sample to a remote server before making a decision. Local processing reduces latency, reduces communication traffic, and allows the device to continue operating even when a network connection is unstable.

Field-programmable gate arrays are useful for this type of work because they allow a complete embedded platform to be assembled from configurable logic. A soft-core processor can execute control firmware, while standard or custom hardware blocks can handle sensor interfaces and time-critical signals. This project uses that idea to build a compact multi-modal anomaly detector around the MicroBlaze processor inside the FPGA fabric.

The system is called FMEAD-FPGA, which stands for FPGA Multi-Modal Edge Anomaly Detector. It was implemented for the Reconfigurable Embedded Systems course at the Faculty of Informatics, University of Debrecen. The aim is not to claim a complex artificial intelligence system, but to show a complete engineering implementation that combines hardware design, sensor integration, bare-metal firmware, local display control, and a browser-based dashboard.

2. Project Motivation and Objectives

The motivation of the project is to build a visible and testable embedded system that connects real sensors to an FPGA-based processing platform. A useful course project should not only compile in Vivado and Vitis; it should also show live behavior on real hardware. For this reason, the prototype combines several types of input data and several forms of output. The board can be moved, heated, exposed to a clap or knock, and observed through LEDs, the seven-segment display, the OLED module, and the Wi-Fi dashboard.

The main objectives of the implementation were the following:

1. Implement a complete MicroBlaze-based system on the Nexys 4 DDR board using Vivado IP integrator.
2. Connect and read heterogeneous sensors through SPI, I2C, XADC, and AXI GPIO interfaces.
3. Process sensor data locally using integer arithmetic and simple state machines suitable for a bare-metal soft-core processor.
4. Classify motion, environmental, FPGA temperature, and acoustic events into discrete event labels and risk levels.
5. Display the system state locally on LEDs, the seven-segment display, and the optional Pmod OLED RGB module.
6. Serve a browser dashboard through the PmodESP32 configured as a Wi-Fi access point and HTTP server.

7. Keep the design realistic by documenting limitations and avoiding unsupported claims such as speech recognition or advanced artificial intelligence.

3. Hardware Platform

3.1 Digilent Nexys 4 DDR Board

The target board is the Digilent Nexys 4 DDR, based on a Xilinx Artix-7 FPGA. The board is suitable for reconfigurable embedded systems because it provides a programmable FPGA device, a 100 MHz clock source, switches, buttons, LEDs, an eight-digit seven-segment display, Pmod connectors, JTAG programming, and several on-board peripherals. These resources allowed the system to be developed and demonstrated without a custom PCB.

In this project, the FPGA fabric hosts the MicroBlaze processor, the local memory system, the AXI interconnect, the sensor interface IP blocks, the display GPIO blocks, the XADC Wizard, and the custom PDM microphone monitor block. The external modules are attached through Pmod connectors, while the on-board ADXL362 accelerometer and the board displays remain directly connected to the FPGA pins.

3.2 MicroBlaze Soft-Core Processor

The MicroBlaze processor is used as the central controller. It reads sensor data, executes the classification logic, updates the display state, and handles HTTP requests from the ESP32 module. The firmware runs without an operating system. This bare-metal approach makes the program structure simple and predictable, but it also means that timing must be managed carefully inside the main loop. Long blocking waits can freeze local outputs if they are not written carefully.

3.3 Sensor and Output Modules

The implemented hardware combines motion, environment, internal FPGA health, and acoustic event detection. The ADXL362 accelerometer provides three-axis motion data. The BME280 provides ambient temperature, relative humidity, and pressure. The XADC provides FPGA die temperature. The PDM microphone path provides a simple acoustic level or sample interface for detecting sudden impulsive events. The PmodESP32 provides Wi-Fi communication, and the OLED and seven-segment displays provide local status output.

3.4 Prepared PmodBT2 Bluetooth Interface

In addition to the Wi-Fi dashboard, the Vivado hardware design includes a PmodBT2 block as a prepared optional communication interface. The purpose of this block is to make the project extensible beyond Wi-Fi. In a future version, Bluetooth serial communication could be used for lightweight telemetry, local configuration commands, or a simple mobile monitoring interface when Wi-Fi access point mode is not required.

The PmodBT2 interface was included in the hardware architecture, but it was not used as the main validated output channel in the final demonstration. For this reason, the report treats it as a prepared extension rather than as a fully tested project feature. The final tested communication path remains the PmodESP32 Wi-Fi dashboard.

Table 1. Hardware components and interfaces used in the implemented prototype.

Component	Interface	Purpose in the project
Nexys 4 DDR / Artix-7	FPGA fabric	Hosts the MicroBlaze system, AXI peripherals, XADC Wizard, GPIO blocks, and custom microphone logic.
MicroBlaze	AXI master	Executes the bare-metal firmware and controls acquisition, classification, display updates, and HTTP service.
ADXL362 accelerometer	AXI Quad SPI	Measures X, Y, and Z acceleration for motion, vibration, impact, and possible fall candidate detection.
BME280 environmental sensor	AXI IIC	Measures ambient temperature, humidity, and pressure.
XADC	XADC Wizard / XSysMon	Measures internal FPGA die temperature.
PDM microphone monitor	Custom logic plus AXI GPIO	Provides a simple acoustic signal path for QUIET and CLAP / IMPACT event detection.
PmodESP32	UART / AT commands	Creates the Wi-Fi access point and serves the web dashboard.
Seven-segment display and LEDs	AXI GPIO	Provide local status and event indication.
Pmod OLED RGB	SPI and GPIO through Pmod IP	Shows compact local event and score information.
PmodBT2	UART / GPIO through Pmod IP	Prepared optional Bluetooth communication interface for future local telemetry or serial monitoring. It was not part of the final validated demonstration.



Figure 1. Final integrated hardware prototype.

This figure should prove that the system was tested as an integrated physical prototype, not only as a simulated block design. It should show that the board, local displays, ESP32 Wi-Fi module, and dashboard were used together during the demonstration.

4. Vivado Hardware Design

4.1 Block Design Overview

The hardware system was assembled in Vivado using an IP integrator block design. The design is centered on the MicroBlaze processor and an AXI interconnect. The interconnect exposes the sensor controllers and display peripherals to the processor as memory-mapped devices. A clocking wizard generates the internal clock, and a processor system reset block distributes synchronized reset signals to the processor and AXI peripherals.

The uploaded block design shows the main implemented structure: the MicroBlaze core, local memory, AXI interconnect, AXI Quad SPI for the ADXL362, AXI IIC for the BME280, XADC Wizard, Pmod OLED RGB IP, AXI GPIO blocks for buttons, LEDs, switches, seven-segment anodes and segments, UART interfaces, PmodESP32, and the PDM microphone monitor path. This confirms that the project is built as a real SoC-style embedded system inside the FPGA rather than as a single isolated peripheral.

4.2 Main IP Blocks

Table 2. Main Vivado IP blocks and their purpose in the FMEAD-FPGA block design.

IP block	Role	Verification point
microblaze_0	Soft-core processor running the firmware	Must be present in the block design and exported to Vitis.
microblaze_0_local_memory	Instruction and data memory for the processor	Must be large enough for the ELF, static buffers, and dashboard strings.
microblaze_0_axi_periph	AXI interconnect	Must route MicroBlaze transactions to all peripherals.
axi_quad_spi_adxl362	SPI controller for ADXL362	Must expose correct SPI pins and match the device ID symbol used in firmware.
axi_iic_bme280	I2C controller for BME280	Must match the base address or device ID used by the firmware.
xadc_wiz_0	FPGA die temperature monitor	Must be available to the standalone XSysMon driver.
PmodESP32_0	Wi-Fi module interface	Must expose the UART used for ESP32 AT commands.
PmodOLEDRgb_0	OLED display interface	Must provide the SPI and control GPIO signals for the OLED.
axi_gpio_7seg_an / axi_gpio_7seg_seg	Seven-segment display control	Must map to the anode and segment top-level ports.
axi_gpio_leds / axi_gpio_switches / axi_gpio_buttons	Local board I/O	Must map to the board LEDs, switches, and buttons.
pdm_mic_monitor_0 and AXI GPIO microphone path	PDM microphone front-end and software-readable sample or level output	Must generate microphone clock/control and expose a valid data path to MicroBlaze.
PmodBT2_0	Prepared optional Bluetooth serial interface	Present in the hardware design but not used as the final validated telemetry interface.

4.3 Clocking and Reset

The design uses the board clock as the input to the clocking wizard. The generated internal clock drives the MicroBlaze processor, the AXI interconnect, and the AXI peripherals. A processor system reset block ensures that the processor and peripherals leave reset only after the generated clock is stable. This is important because peripherals such as the SPI, I2C, UART, and GPIO controllers must share a consistent clock and reset domain to avoid unreliable behavior during boot.

4.4 Implementation Reports and Hardware Verification

The implemented hardware was verified using Vivado reports generated from the implemented design. The final top-level design is mb_edge_wrapper and the target device is the Xilinx Artix-7 XC7A100T in the xc7a100tcsg324-1 package. The implementation report shows that the design

reached the Routed state, which means that synthesis, placement, and routing were completed successfully.

The clock report confirms that the board system clock is constrained with a 10.000 ns period, corresponding to 100 MHz. The clocking wizard generates additional internal clocks, including 100 MHz clock outputs for the main logic and a 50 MHz generated clock output. The MicroBlaze system, AXI peripherals, and most sensor interface blocks are therefore driven from a controlled clocking structure rather than from unrelated free-running clocks.

Timing analysis reports no unclocked register or latch pins and no unconstrained internal endpoints. The worst setup slack reported for the implemented design is positive, with the most critical listed setup slack approximately 1.697 ns and total violation equal to 0.000 ns. This indicates that the main routed timing paths meet the implementation timing requirements. The report also lists missing input and output delay constraints for several board-level I/O ports. In this educational prototype, these warnings are expected because several external interfaces are simple board-level GPIO, SPI, I2C, UART, and Pmod signals rather than fully constrained high-speed source-synchronous interfaces.

5. XDC Pin Constraints and External Connections

5.1 Role of the XDC File

The Xilinx Design Constraints file defines the physical mapping between the top-level design ports and the actual FPGA package pins on the Nexys 4 DDR board. It also defines the I/O standard, normally LVCMOS33 for the board I/O and Pmod pins used in this project. The XDC file is therefore a critical part of the implementation. A correct block design can still fail on real hardware if a signal is mapped to the wrong pin or if a bidirectional interface is constrained incorrectly.

The final Vivado I/O report shows that the implemented design uses 90 user I/O pins. These pins include the board switches, LEDs, buttons, seven-segment display pins, ADXL362 SPI lines, BME280 I2C lines, Pmod OLED RGB pins, PmodESP32 pins, PmodBT2 pins, and the microphone interface. The BME280 I2C lines are configured with LVCMOS33 and pull-up behavior, which is important for a reliable I2C bus.

The exact physical pin names should always be copied from the final XDC file or from Vivado report_io, not from memory. This report focuses on the functional signal groups because incorrect package pin documentation would make future debugging harder.

Table 3. Main signal groups that must be verified in the XDC file before final submission.

Signal group	Function	Verification action
Clock and reset	Provide system timing and reset input	Check clock pin, period constraint, and reset polarity in the XDC file.
Switches and LEDs	Enable detector state and display local status	Check SW0 to SW15 and LED0 to LED15 mappings.
Push buttons	Optional user inputs and reset or control functions	Check active polarity and GPIO direction.
Seven-segment display	Shows local event mnemonics and numeric values	Check segment and anode polarity because the display is multiplexed.
ADXL362 SPI	Reads accelerometer samples	Check MOSI, MISO, SCLK, CS, and interrupt lines.
BME280 I2C	Reads ambient temperature, humidity, and pressure	Check SDA, SCL, connector location, and pull-up resistors.
Pmod OLED RGB	Displays compact local status	Check SPI and control pins against the Pmod connector used.
PmodESP32	Provides Wi-Fi dashboard communication	Check UART TX/RX direction and any enable/reset GPIO.
PDM microphone	Provides acoustic event signal path	Check microphone data, clock, and left/right select pins.

5.2 Practical Pin Debugging

Pin assignment errors are common in FPGA projects because the logical signal name and the physical connector pin must both be correct. For SPI, swapping MOSI and MISO prevents the sensor from returning the correct device ID. For I2C, missing pull-up resistors or an incorrect Pmod header mapping prevents acknowledgement from the slave device. For the seven-segment display, wrong segment polarity or anode polarity can produce unreadable digits even when the firmware writes the correct bit pattern.

Before submission, the XDC file should be checked against the schematic or the official Nexys 4 DDR reference manual. The `report_io` output should be saved because it provides a clear list of top-level ports, package pins, and I/O standards used by the implemented design.

6. Firmware Architecture in Vitis

6.1 Bare-Metal Firmware Structure

The firmware is a bare-metal C application built in Vitis using the standalone board support package generated from the exported Vivado hardware platform. There is no operating system, no thread scheduler, and no dynamic memory allocation. The firmware is organized as a deterministic main

loop that repeatedly updates the sensors, classifiers, local outputs, OLED display, and ESP32 HTTP service.

This structure is appropriate for the project because the processing tasks are small and periodic. It also makes the system easier to debug: each peripheral is initialized in a fixed order, and status messages can be printed during boot to identify which module is working and which module failed.

6.2 Initialization Sequence

At startup, the firmware initializes the GPIO peripherals, seven-segment display state, ADXL362 SPI driver, BME280 I2C driver, XADC driver, microphone GPIO path, OLED module, and ESP32 communication interface. Each initialization stage should be visible in the UART boot log. If the OLED is not detected, the system can continue without it because the OLED is not required for core sensing. If a main sensor is missing, the dashboard should show that the corresponding module is offline instead of reporting invalid values as normal measurements.

The ESP32 setup is performed through AT commands. The module is configured as a Wi-Fi access point and simple HTTP server. The firmware sends the dashboard page for GET /, a JSON telemetry response for GET /data, and a page selection response for dashboard tab synchronization. This approach avoids the need for an operating system or TCP/IP stack on MicroBlaze; the ESP32 handles the Wi-Fi side, while MicroBlaze sends and receives command strings through UART.

6.3 Cooperative Scheduling

The most important firmware design choice is cooperative scheduling. The MicroBlaze cannot block for long periods while waiting for the ESP32, because the seven-segment display must be refreshed continuously and sensor updates should continue. Therefore, waiting loops are written so that they also call local service functions. This keeps the seven-segment display alive during UART transmission and prevents the user interface from freezing while the dashboard page is being served.

The same principle applies to OLED updates. The OLED should not be updated on every loop iteration because screen operations are slower than simple GPIO writes. Instead, the OLED summary can be refreshed at a lower rate while the fast seven-segment scan continues in the background of the main loop.

6.4 Memory Safety

The firmware avoids dynamic memory allocation. Buffers are static and fixed in size. This is important on a small MicroBlaze system because memory overflows can corrupt the stack, the dashboard buffer, or other global variables. The HTTP page buffer should be sized according to the actual HTML response generated, and all writes to the response buffer should be performed through bounds-checked helper functions.

A previous source of instability in this type of project is an HTML page that grows beyond the available buffer or beyond the local memory available to the MicroBlaze system. The correct solution

is not to keep adding dashboard elements without control. The safe solution is to keep the page efficient, use bounds checking, reduce repeated HTML, and confirm that the final ELF fits inside the available instruction and data memory.

7. Sensor Integration and Detection Logic

7.1 ADXL362 Motion Detection

The ADXL362 accelerometer is used to detect normal movement, repeated vibration, abrupt impact, and possible fall candidate behavior. The firmware reads signed X, Y, and Z acceleration values through the AXI Quad SPI controller. A resting baseline is estimated during calibration and then updated slowly only when the board is considered stable. This prevents the reference position from drifting during active movement.

The motion pipeline uses two main quantities. The first is a motion score, computed from the difference between the filtered acceleration and the slow reference. This value indicates how far the board is from its normal position. The second is a delta or jerk proxy, computed from the difference between consecutive filtered samples. This value indicates how abruptly the board is moving.

The classifier is designed to separate several practical cases. Normal handling or slow movement should produce MOVING rather than IMPACT. Repeated moderate changes increase a vibration accumulator and can produce VIBRATION. A sudden high delta produces IMPACT. A possible FALL CANDIDATE requires a strong transient together with a significant orientation change. This logic is still threshold-based, but the use of hysteresis, persistence counters, event hold time, and a slowly updated baseline makes it much more stable than a single-threshold detector.

A future revision should expose a calibration mode in the dashboard so that the user can tune sensitivity for normal motion, abrupt motion, vibration, fall candidate detection, and impact detection without recompiling the firmware. This would make the same hardware usable in different demonstration environments.

7.2 BME280 Environmental Monitoring

The BME280 provides ambient temperature, relative humidity, and pressure. The sensor does not output final physical units directly. Its raw measurements must be compensated using calibration constants stored inside the sensor. The firmware reads the calibration registers during initialization and applies compensation formulas in integer arithmetic. Temperature is represented in centi-degrees Celsius, humidity in centi-percent, and pressure in pascals or hectopascals for display.

Environmental events are generated from thresholds. Ambient temperature can produce NORMAL, AMBIENT WARM, or AMBIENT HOT. Humidity can produce HUMIDITY LOW or HUMIDITY HIGH. Pressure is monitored and displayed but should not be overinterpreted unless the test environment is controlled. The environmental path is useful for showing that the FPGA system can

integrate a real external I2C sensor and present slow-changing physical data alongside faster motion and acoustic events.

7.3 XADC FPGA Temperature Monitoring

The XADC is used to read the internal FPGA die temperature. This value is different from ambient temperature because the die temperature depends on FPGA activity, airflow, and board heating. Monitoring it is useful in an embedded FPGA project because a system can appear stable at the environmental sensor while the FPGA fabric itself becomes warmer during operation.

The firmware converts the XADC raw value into a temperature value using the standard conversion formula for the Xilinx 7-series XADC. The resulting FPGA temperature is used in the environmental classifier and displayed on the dashboard. A high difference between FPGA temperature and ambient temperature can be reported as a thermal gap event.

7.4 PDM Microphone Acoustic Event Detection

The acoustic module is intentionally limited to event detection. It is used to detect quiet background conditions and short impulsive events such as a clap, knock, or impact near the microphone. It does not recognize words, identify speakers, transcribe speech, or classify music. This distinction is important because the MicroBlaze configuration used in this project is not intended for heavy audio analysis.

The microphone path provides a software-readable acoustic signal through AXI GPIO. The firmware estimates a background level and compares new samples or short frames against that baseline. A CLAP / IMPACT event requires a clear peak, energy increase, or sudden jump above the adaptive baseline. A hold time keeps the event visible long enough for the dashboard, and a refractory period prevents one clap from being counted repeatedly.

The current validated state is focused on QUIET and CLAP / IMPACT. A realistic extension would add a CONTINUOUS SOUND state to indicate sustained sound energy caused by speech activity, music, or continuous noise. This would still not be speech recognition or music recognition; it would only mean that the sound level remains above the background for a longer time. Such a feature is appropriate for MicroBlaze because it can be implemented with integer energy, peak, crest factor, zero-crossing rate, and duration counters rather than a heavy audio model.

7.5 Event Fusion and Risk Levels

Each modality produces a local event and a local risk level. The global system state is selected from the most severe active modality. For example, if motion is NORMAL and environment is NORMAL but the acoustic detector reports CLAP / IMPACT, the global state should show the acoustic event. If the BME280 reports AMBIENT HOT, the global system should report that environmental condition even if the board is not moving.

The event fusion logic is intentionally simple and transparent. This is better for an engineering report than a hidden or unexplained classifier. The dashboard and local displays should make it clear which module produced the active event and what risk level was assigned.

Table 4. Event classes and risk levels used by the multi-modal detector.

Event class	Source	Typical meaning	Suggested risk level
NORMAL	All modules	No significant event detected	LOW
MOVING	ADXL362	Board orientation or position changed from baseline	LOW or MEDIUM
VIBRATION	ADXL362	Repeated moderate movement or shaking	MEDIUM
IMPACT	ADXL362	Short high-delta motion event	HIGH
FALL CANDIDATE	ADXL362	Large change with possible orientation transition	HIGH
AMBIENT WARM / HOT	BME280	Ambient temperature crossed warning or high threshold	MEDIUM or HIGH
FPGA WARM / HOT	XADC	FPGA die temperature crossed warning or high threshold	MEDIUM or HIGH
HUMIDITY LOW / HIGH	BME280	Relative humidity outside selected operating range	MEDIUM
CLAP / IMPACT	PDM microphone	Short impulsive acoustic event	HIGH
SENSOR OFFLINE	Any module	A required sensor did not initialize or did not return valid data	MEDIUM or HIGH

8. Output Interfaces and Dashboard

8.1 ESP32 Wi-Fi Dashboard

The browser dashboard is served through the PmodESP32. The ESP32 is controlled by AT commands and configured as a Wi-Fi access point. After connecting to the board network, the user opens the board address in a browser. The dashboard provides a compact view of global status and separate pages for motion, environment, acoustic, and system information.

The dashboard uses a JSON telemetry endpoint to update live values without reloading the full page. This is important because sending the full dashboard repeatedly would waste UART bandwidth and increase the risk of blocking the local outputs. The dashboard can poll the /data endpoint at a

moderate interval, while the firmware continues refreshing the seven-segment display and updating the classifiers.

8.2 Seven-Segment Display and LEDs

The seven-segment display gives immediate local feedback even when no browser is connected. It is driven by AXI GPIO outputs and multiplexed in software. Each scan step activates one anode and writes the corresponding segment pattern. Because the display is multiplexed, the scan function must be called frequently. If the firmware waits too long inside a UART loop, the display flickers or freezes. The final design avoids this by refreshing local outputs during communication waits.

The LEDs provide an additional local indication of system status. They can show enable state, active module state, score level, or anomaly status depending on the firmware configuration. The LEDs are useful during debugging because they still work even if the OLED or Wi-Fi dashboard is not available.

8.3 Pmod OLED RGB Display

The OLED display provides a small local summary of the system state. Because the screen is limited in size, it should not attempt to reproduce the whole dashboard. A good OLED layout shows only the project label, global event, risk level, and a few compact numeric indicators such as motion score, acoustic score, and environmental score.

The OLED is treated as an optional output. This is a practical design decision. If the OLED does not initialize because of wiring, pin mapping, or module issues, the rest of the system should still run. Optional output modules should not prevent the core sensors and dashboard from working.

8.4 Prepared Bluetooth Telemetry Interface

The PmodBT2 block was kept in the hardware design as a prepared interface for future Bluetooth-based telemetry. Its practical purpose is to provide a second local communication path, separate from the ESP32 Wi-Fi access point. In a future version, Bluetooth could be used to stream compact status values, receive simple configuration commands, or connect to a phone or laptop serial terminal without opening the web dashboard.

This interface was not used as the primary validated output in the final demonstration, so the report does not claim Bluetooth dashboard functionality. It is included as an extensibility point and as evidence that the hardware platform was prepared for more than one communication method.

9. Testing Results

9.1 Integrated Prototype Test

The first test is the integrated prototype test. The board should be powered, programmed with the bitstream and ELF, and observed through the serial terminal, local displays, and dashboard. A

successful integrated test confirms that the hardware platform boots, the firmware starts, the ESP32 access point is created, and at least the Home dashboard updates live data.

The final prototype photograph should be inserted as Figure 2. It should be clear enough to show the board, connected modules, local displays, and dashboard device. This image is useful because it proves physical integration. The caption should mention only what is visible and technically relevant.

9.2 Motion Dashboard Test



Figure 3. Motion dashboard during a board movement or stable face-up state.

This figure proves that the ADXL362 accelerometer is read by the MicroBlaze firmware and that motion values are sent correctly to the dashboard. A stable trace supports the NORMAL state, while a visible transient supports MOVING, VIBRATION, or IMPACT classification depending on the test.

9.3 Environmental Dashboard Test

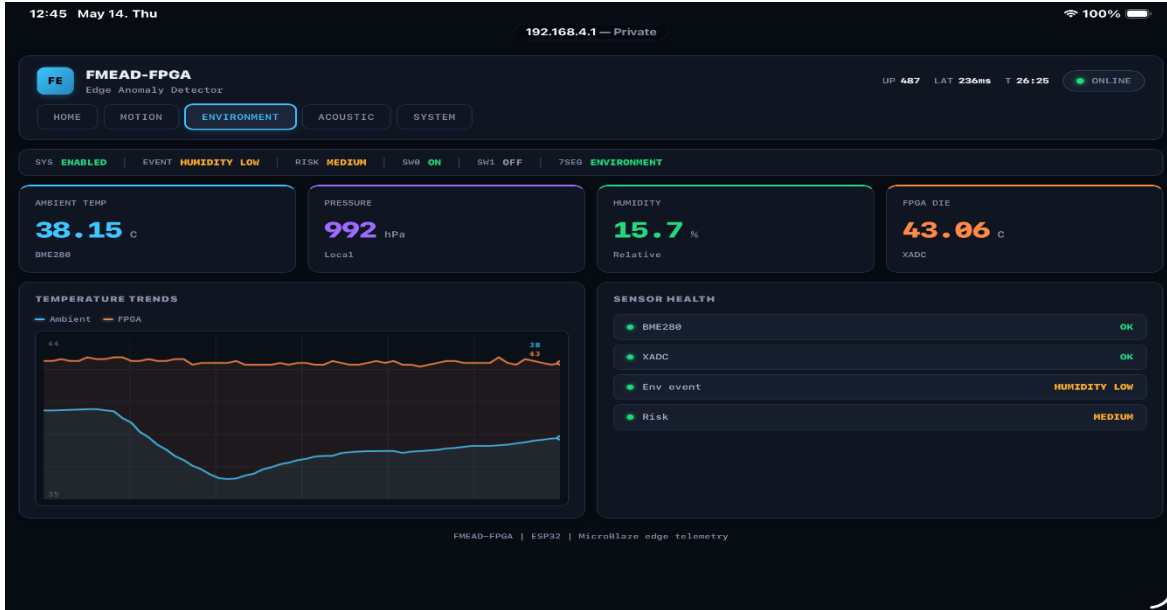


Figure 4. Environmental dashboard showing BME280 and XADC measurements

This figure proves that the BME280 and XADC paths are both active. The BME280 validates the external I2C sensor path, while the XADC validates internal FPGA temperature monitoring. If a heat source is used carefully near the BME280, the ambient temperature trend should change while the FPGA die temperature changes more slowly.

9.4 Acoustic Dashboard Test

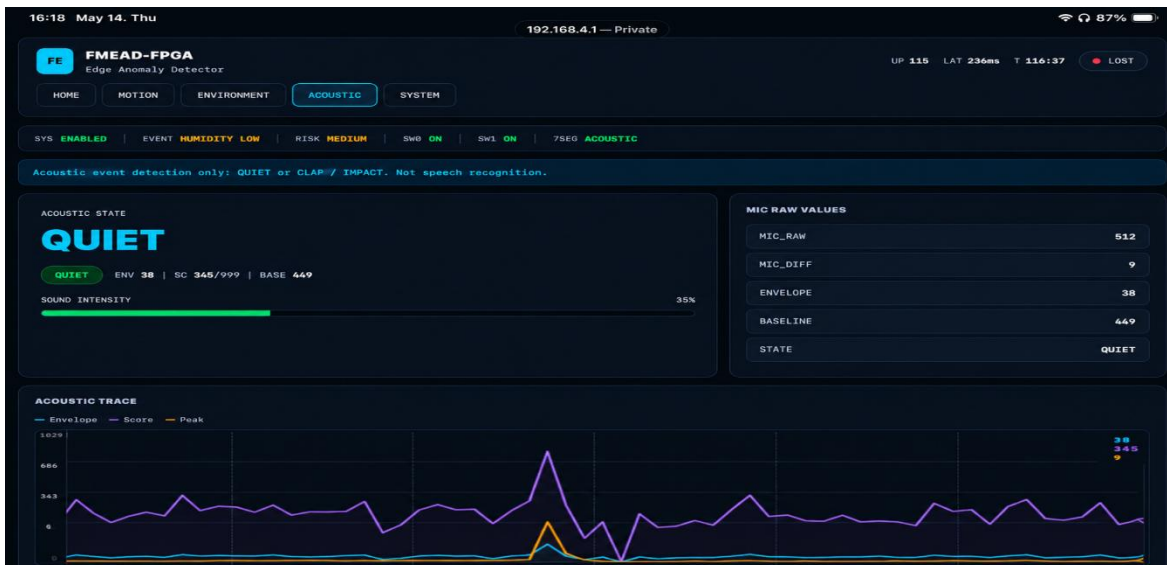


Figure 5. Acoustic event test showing a CLAP / IMPACT event.

This figure proves that the microphone path and acoustic state machine can detect a short impulsive event. The explanation must state clearly that the module performs simple event detection only. It should not be presented as speech recognition or music classification.

9.5 Home Dashboard Test

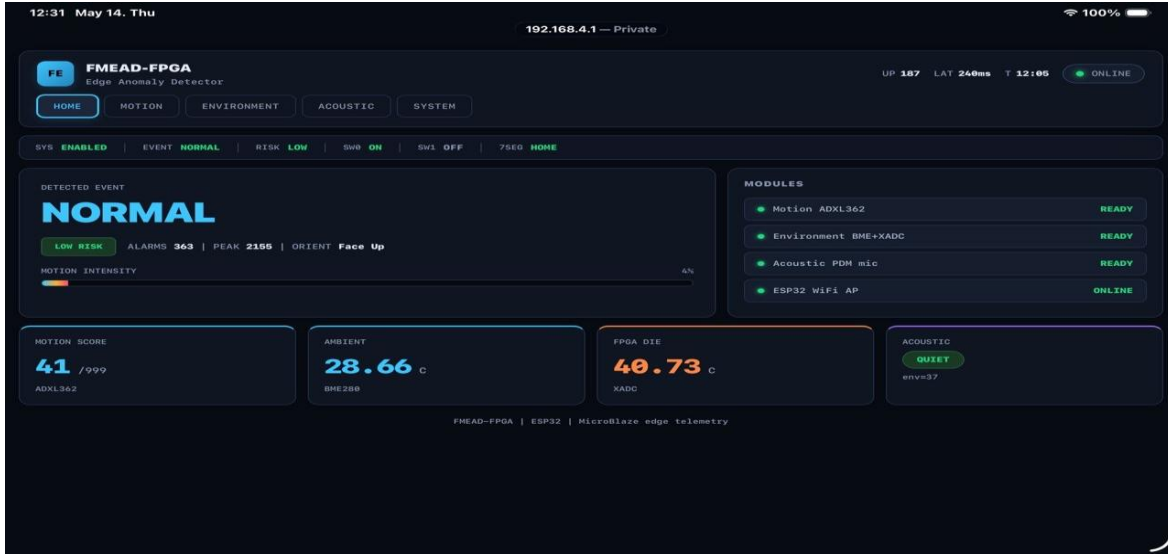


Figure 6. Home dashboard showing global event status, risk level, and module readiness.

This figure proves the global fusion behavior of the system. It should show that a local module event can be promoted to the global event field and that the dashboard reports whether the main hardware modules are ready.

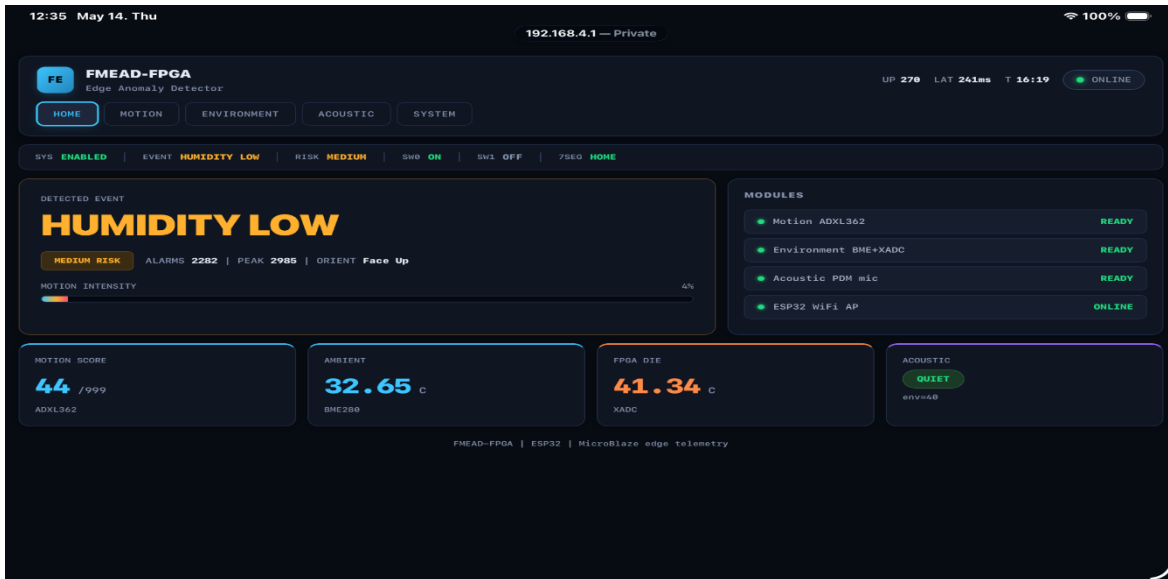


Figure 7. Home dashboard response during a HUMIDITY LOW event.

This figure shows the system detecting a low humidity condition from the BME280 environmental sensor. The global event field changes to HUMIDITY LOW and the risk level is classified as MEDIUM. The motion score remains low, the acoustic state remains QUIET, and all modules are reported as ready. This confirms that the environmental classifier can trigger a global event independently from the motion and acoustic modules.

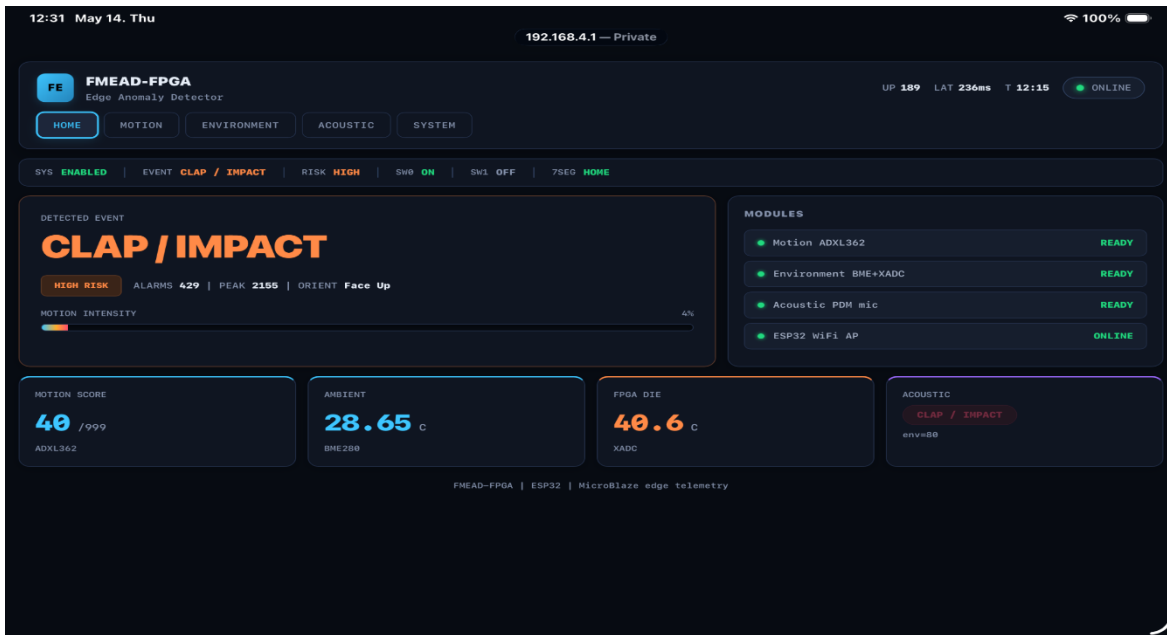


Figure 8. Home dashboard response during a CLAP / IMPACT acoustic event.

This figure shows the response of the acoustic detector after a short impulsive sound was generated near the microphone. The system reports CLAP / IMPACT with HIGH risk, while the motion score and environmental values remain normal. This result confirms that the PDM microphone pipeline can detect a sudden acoustic event and propagate it to the global dashboard state without requiring speech recognition or frequency analysis.

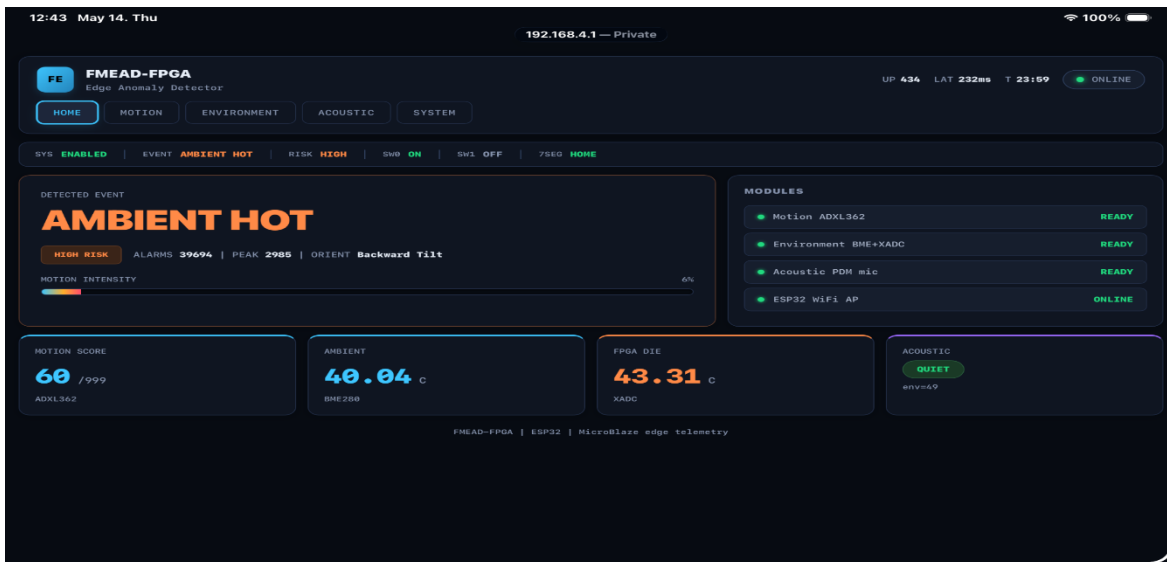


Figure 9. Home dashboard response during an AMBIENT HOT event.

This figure shows the environmental classifier responding to a high ambient temperature condition. The BME280 reports an ambient temperature of approximately 40.04 °C, while the XADC reports an FPGA die temperature of approximately 43.31 °C. The dashboard classifies the event as AMBIENT

HOT with HIGH risk. This confirms that the firmware correctly converts sensor readings into engineering units and applies threshold-based classification for environmental monitoring.

9.6 OLED and Seven-Segment Local Display Test



Figure 10. Local display test showing the OLED and seven-segment output.

This figure proves local human-machine interface functionality. It is especially important because an embedded system should still provide useful feedback when a Wi-Fi client is not connected.

9.7 Test Summary

Table 5. Functional tests that should be completed before the final demonstration.

Test	Expected observation	Result to verify before submission
Boot and initialization	UART log reports successful initialization or clear offline status for each module	No unexplained boot freeze or silent failure.
ADXL362 motion	Dashboard and local display react to movement, vibration, or impact	Motion values return to stable baseline when board is still.
BME280 environment	Ambient temperature, humidity, and pressure are displayed	Values are realistic for the room and change under controlled test.
XADC temperature	FPGA die temperature is displayed	Value is plausible and does not report impossible numbers.
PDM microphone	Clap or knock triggers CLAP / IMPACT briefly	Normal background remains QUIET after adaptation.
ESP32 dashboard	Browser loads Home page and /data updates live values	Dashboard does not corrupt memory or freeze local displays.
Seven-segment display	Digits refresh continuously	No visible freeze during HTTP response.
OLED display	Compact state is shown if OLED initializes	Failure of OLED does not stop the rest of the system.

10. Problems Faced and Solutions

10.1 Sensor Bring-Up Issues

The most common problem during sensor bring-up is that a device does not return the expected ID register. For SPI devices, this often indicates that MOSI, MISO, SCLK, or chip select is mapped incorrectly or that the SPI mode is wrong. For I2C devices, it often indicates missing pull-up resistors, wrong slave address, wrong connector pins, or an incorrect I2C controller base address in the firmware.

The practical solution is to test each sensor alone before integrating it into the full dashboard firmware. A small standalone test should first read only the device ID. Once the ID is correct, the calibration or measurement registers can be read. This method avoids confusing a hardware pin problem with a software classifier problem.

10.2 ESP32 and HTTP Stability

The ESP32 AT command interface is convenient but sensitive to command order, timeouts, and connection state. The firmware must send a stable initialization sequence and wait long enough for the access point and server to start. The server should also handle unexpected browser requests safely, including favicon or repeated polling requests.

Another important problem is dashboard size. If the HTML response grows beyond the allocated buffer or beyond MicroBlaze memory capacity, the application can crash or produce corrupted output. The solution is to keep the dashboard efficient, use fixed-size static buffers, check every write against the buffer limit, and avoid adding repeated or unnecessary dashboard sections.

10.3 Seven-Segment Flicker During Communication

A multiplexed seven-segment display needs continuous refresh. During early testing, long UART waits can make the display flicker or freeze on one digit. The solution is to call the display refresh function inside wait loops and during long ESP32 send operations. This keeps the local display responsive even while the browser page is being transmitted.

10.4 Acoustic Detector Sensitivity

The acoustic detector must be tuned carefully. If thresholds are too low, normal background sound or keyboard noise can trigger false CLAP / IMPACT events. If thresholds are too high, real claps are missed. The implemented approach uses an adaptive baseline, a peak or energy jump condition, a short hold time, and a refractory period. This is a suitable compromise for a simple event detector on a MicroBlaze system.

10.5 Local Memory Pressure

A MicroBlaze design with local memory can run out of instruction or data memory when the firmware becomes large. Dashboard HTML, string constants, tables, and display code can quickly increase the ELF size. The solution is to reduce repeated text, keep HTML compact, avoid unused features, and confirm the memory map and linker script after every major change. If the application no longer fits, the block design memory configuration must be changed rather than forcing unsafe firmware changes.

11. Limitations

The prototype has clear limitations that must be stated honestly. First, the acoustic module is a simple event detector. It does not understand speech, identify speakers, transcribe words, or recognize music. It can detect impulse-like events and, in a future version, could detect sustained sound activity, but it should not be presented as audio understanding.

Second, the system currently uses threshold-based and state-machine logic, not a trained machine learning model. This makes the behavior transparent and suitable for debugging, but it also means that thresholds may need tuning for a different board position, sensor placement, room, or demonstration style.

Third, the firmware is mostly polling-based. This is acceptable for a course prototype, but a stronger real-time version should use interrupts for selected peripherals, especially accelerometer data-ready events and microphone sample-ready events. Fourth, the dashboard is limited by MicroBlaze memory and UART bandwidth. Adding large JavaScript, many plots, or repeated HTML sections can cause memory pressure.

Fifth, the PmodBT2 interface is present in the hardware design, but Bluetooth telemetry was not validated as part of the final demonstration. It is therefore documented as a prepared extension. Finally, the system does not store long-term logs. Alarm counters and traces are volatile and reset when the board restarts.

12. Future Work

Future work should improve the project in directions that match both the FPGA platform and the limits of MicroBlaze. The first priority is to improve motion sensitivity. The dashboard should expose a calibration page where the user can tune thresholds for NORMAL movement, MOVING, VIBRATION, IMPACT, and FALL CANDIDATE detection. This would make the detector easier to adapt to different physical setups without recompiling the firmware.

The acoustic path can also be extended. The current demonstration focuses on QUIET and CLAP / IMPACT. A future version can add a CONTINUOUS SOUND state for sustained sound activity such as someone speaking, music playing, or continuous background noise. This should still be described as level-based acoustic activity detection, not word recognition or music recognition. Suitable

MicroBlaze features include integer energy, peak level, crest factor, zero-crossing rate, duration above threshold, and adaptive baseline tracking.

The dashboard should be improved without making it too large for the MicroBlaze memory system. Useful improvements include a smaller page structure, better separation between Home, Motion, Environment, Acoustic, and System pages, a calibration page, exportable JSON logs, and clearer module health indicators. The web interface should remain efficient because the ESP32 AT link and MicroBlaze local memory are limited resources.

A more ambitious research direction is to add lightweight machine learning at the edge. On the current MicroBlaze system this should be done carefully. Realistic options include a small decision tree, a linear classifier, a tiny k-nearest-neighbor table, or a fixed-point logistic regression model trained offline and then converted to integer C code. Larger neural networks, speech models, or FFT-heavy audio classification should not run directly in the current MicroBlaze loop unless they are strongly simplified or moved into custom FPGA logic.

The FPGA fabric also makes hardware acceleration possible. Selected functions such as accelerometer filtering, microphone energy calculation, PDM-to-PCM conversion, or a small feature extractor could be moved into custom AXI peripherals. This would reduce the firmware load and use the reconfigurable fabric more effectively than a pure software solution.

Table 6. Main limitations and possible future improvements.

Limitation	Possible improvement
Simple acoustic event detection only	Add improved PDM-to-PCM filtering and a stronger impulse detector while keeping speech recognition out of scope.
Polling-based firmware loop	Use interrupts for selected peripherals, especially accelerometer data-ready and microphone sample-ready signals.
Dashboard memory pressure	Split the dashboard into smaller responses and reduce repeated HTML and JavaScript.
No persistent logging	Add SD card or external memory logging for event history.
Thresholds tuned for one setup	Add calibration mode and store tuned thresholds for each demonstration environment.
Limited local display area	Use a larger display or dedicated graphical display controller.

13. Conclusion

FMEAD-FPGA demonstrates a complete reconfigurable embedded system implemented on the Nexys 4 DDR board. The project combines a MicroBlaze soft-core processor, AXI peripherals, real sensors, local displays, prepared communication interfaces, and a Wi-Fi dashboard. It shows how an FPGA platform can be used not only for isolated digital logic, but also for a complete sensor-based embedded application.

The strongest engineering result is the integration of several heterogeneous modules into one working prototype. The ADXL362 path provides motion detection for normal movement, vibration, impact, and possible fall candidate behavior. The BME280 path provides environmental monitoring. The XADC path provides internal FPGA temperature monitoring. The microphone path provides simple acoustic event detection. The ESP32 dashboard, OLED, seven-segment display, and LEDs make the result visible and testable. The PmodBT2 interface was prepared as an optional future communication path.

The Vivado implementation reports show that the design is routed, uses a modest amount of LUT and flip-flop resources, uses block RAM mainly for the MicroBlaze system, and uses no DSP blocks. The timing summary reports positive setup slack and no total violation in the main timing summary. The remaining warnings are typical of a board-level educational prototype and are documented as future cleanup work.

The project also shows practical embedded design discipline. Fixed-size buffers, integer arithmetic, cooperative scheduling, local display refresh during communication waits, and clear offline handling are necessary for a reliable MicroBlaze implementation. The current prototype is not a safety-certified system and does not perform advanced artificial intelligence.

References

- [1] Digilent, Nexys 4 DDR FPGA Board Reference Manual. Digilent Inc. Available: Digilent Reference Documentation. Accessed: May 14, 2026.
- [2] Digilent, Nexys 4 DDR Schematic. Digilent Inc. Available: Digilent Reference Documentation. Accessed: May 14, 2026.
- [3] Digilent, Pmod OLED RGB Reference Manual. Digilent Inc. Available: Digilent Pmod OLEDrgb Documentation. Accessed: May 14, 2026.
- [4] Digilent, Pmod ESP32 Reference Manual. Digilent Inc. Available: Digilent Pmod ESP32 Documentation. Accessed: May 14, 2026.
- [5] Digilent, Pmod Interface Specification. Digilent Inc. This reference supports the explanation of Pmod connectors, external module wiring, and FPGA peripheral expansion.
- [6] Analog Devices, ADXL362: Micropower, 3-Axis, ± 2 g/ ± 4 g/ ± 8 g Digital Output MEMS Accelerometer Datasheet, Rev. G, 2023. The ADXL362 datasheet is important because it defines the SPI interface, output resolution, motion ranges, activity detection functions, and register map used for accelerometer integration.
- [7] Bosch Sensortec, BME280 Combined Humidity and Pressure Sensor Datasheet. Bosch Sensortec GmbH. This reference supports the temperature, humidity, pressure measurement, I2C communication, and compensation formulas used in the environmental sensing module.
- [8] AMD/Xilinx, MicroBlaze Processor Reference Guide, UG984. AMD Adaptive Computing Documentation. This document explains the MicroBlaze soft processor architecture, pipeline options, memory interfaces, AXI connectivity, interrupts, FPU support, and hardware acceleration possibilities used in the embedded design.
- [9] AMD/Xilinx, 7 Series FPGAs and Zynq-7000 SoC XADC Dual 12-Bit 1 MSPS Analog-to-Digital Converter User Guide, UG480, Rev. 1.11, 2022. This reference supports the use of the XADC block for FPGA die temperature monitoring and internal analog sensor reading.
- [10] AMD/Xilinx, Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator, UG994. AMD Adaptive Computing Documentation. This reference supports the block design methodology, AXI interconnect, IP configuration, wrapper generation, and hardware export flow.
- [11] AMD/Xilinx, Vivado Design Suite User Guide: Using Constraints, UG903. AMD Adaptive Computing Documentation. This reference supports the XDC constraints, pin assignments, clock definition, IOSTANDARD settings, pullups, and external port mapping.
- [12] AMD/Xilinx, Vitis Unified Software Platform Documentation: Embedded Software Development. AMD Adaptive Computing Documentation. This reference supports the bare metal C firmware development, BSP generation, driver usage, memory map, and MicroBlaze software build process.

- [13] AMD/Xilinx, AXI GPIO Product Guide, PG144. AMD Adaptive Computing Documentation. This reference supports the use of AXI GPIO for LEDs, switches, buttons, microphone level input, and seven segment display control.
- [14] AMD/Xilinx, AXI Quad SPI Product Guide, PG153. AMD Adaptive Computing Documentation. This reference supports the SPI communication used to interface the MicroBlaze processor with the onboard ADXL362 accelerometer.
- [15] AMD/Xilinx, AXI IIC Bus Interface Product Guide, PG090. AMD Adaptive Computing Documentation. This reference supports the I2C communication used for the BME280 environmental sensor.
- [16] AMD/Xilinx, AXI UART Lite Product Guide, PG142. AMD Adaptive Computing Documentation. This reference supports the UART communication used between MicroBlaze and the ESP32 AT firmware module.
- [17] Espressif Systems, ESP-AT User Guide: ESP32 AT Command Set. Espressif Systems Documentation. This reference supports the WiFi access point mode, TCP server configuration, CIPMUX, CIPSEND, CIPCLOSE, and HTTP communication used in the dashboard.
- [18] Espressif Systems, ESP32 Series Datasheet. Espressif Systems. This reference supports the description of the ESP32 module as a WiFi communication device used to expose the local web dashboard.
- [19] Analog Devices, ADMP421 Omnidirectional Microphone with PDM Digital Output Datasheet. Analog Devices. This reference supports the explanation of the onboard PDM microphone interface, clock, data output, and the need for PDM to PCM conversion in future audio processing.
- [20] Digilent, Nexys 4 DDR Master XDC Constraints File. Digilent Inc. This reference supports the board pin mappings used for clock, reset, switches, LEDs, buttons, seven segment display, ADXL362 SPI, Pmod connectors, and onboard microphone pins.
- [21] Digilent, Pmod BT2 Reference Manual. Digilent Inc. This reference can be included because Bluetooth was prepared as a future or optional communication module, even if it was not fully tested in the final prototype.
- [22] R. G. Lyons, Understanding Digital Signal Processing, 3rd ed. Pearson, 2010. This book is useful for explaining filtering, moving averages, signal energy, noise floor, and basic digital signal processing concepts used in the acoustic detection logic.
- [23] A. V. Oppenheim and R. W. Schaffer, Discrete-Time Signal Processing, 3rd ed. Pearson, 2009. This reference supports the theoretical background of digital filters, sampling, signal analysis, and discrete time processing.
- [24] S. W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing. California Technical Publishing, 1997. This is useful for practical explanations of moving averages, filtering, peak detection, and real time DSP implementation.

Appendix Checklist

8. Open the Vivado block design and run `report_bd_design`. Confirm that there are no critical block design warnings.
9. Run `get_bd_cells` and confirm that the MicroBlaze, AXI interconnect, SPI, I2C, XADC, GPIO, UART, ESP32, OLED, and microphone blocks are present.
10. Run `report_io` or inspect the final XDC file. Confirm that every external pin matches the actual board wiring and Pmod connector used.
11. Confirm that ADXL362 MOSI and MISO are not swapped and that the expected device ID can be read.
12. Confirm that the BME280 responds at the correct I2C address and that SDA and SCL have proper pull-ups.
13. Run `report_utilization` and confirm that LUT, flip-flop, and BRAM usage are within device limits.
14. Run `report_timing_summary` and confirm that timing is met.
15. Export the hardware platform from Vivado and rebuild the Vitis BSP from the final hardware description.
16. Clean and rebuild the firmware. Confirm that there are no important compile warnings, missing prototypes, or linker memory overflow errors.
17. Program the FPGA and run the ELF on the physical board. Check the UART boot log for module initialization results.
18. Connect to the ESP32 access point and open the dashboard. Confirm that the Home page loads and /data updates live values.
19. Test motion by moving or tapping the board. Confirm that the dashboard, seven-segment display, and LEDs react correctly.
20. Test environment by observing BME280 values and, if safe, applying a controlled warm source near the sensor. Confirm that the trend changes realistically.
21. Test the microphone with a single clap or knock. Confirm that CLAP / IMPACT appears briefly and returns to QUIET.
22. Check the OLED and seven-segment display while the dashboard is being loaded. Confirm that the seven-segment display does not freeze during HTTP transmission.